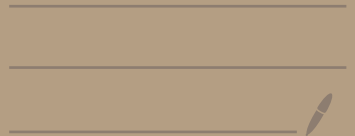


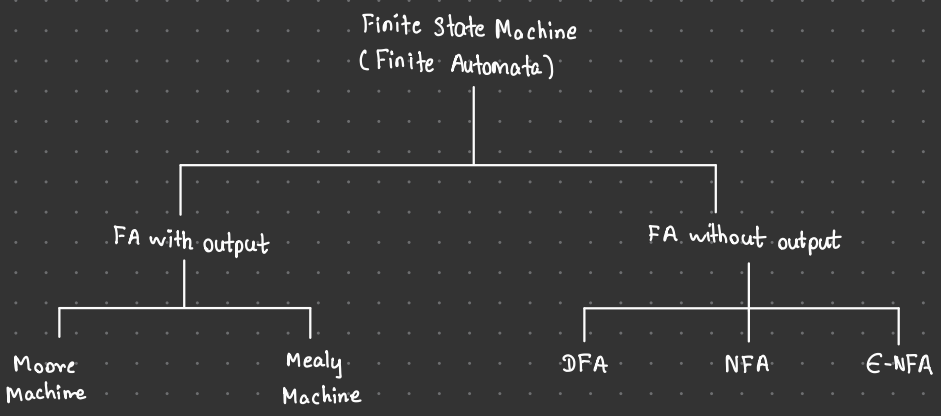
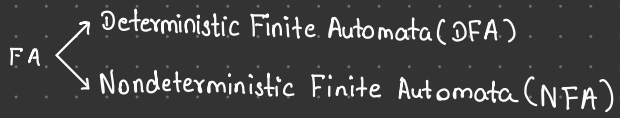
Theory of Computation Notebook

Phyo Thi Khaing

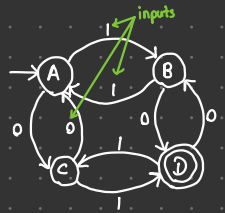


Finite Automata $\Rightarrow$ simplest machine to recognize patterns
 are used to analyze and recognize Natural Language Expressions

Finite Automata aka Finite State Machine



DFA (Deterministic Finite Automata)
 - the simplest model of computation
 - has very limited memory



states: A, B, C, D
 transitions: $\rightarrow$
 $\rightarrow A$: Initial state
 $\odot D$: Final state

$Q = \{A, B, C, D\}$
 $\Sigma = \{0, 1\}$
 $q_0 = A$
 $F = \{D\}$

$\delta =$

	0	1
A	C	B
B	D	A
C	A	D
D	B	C

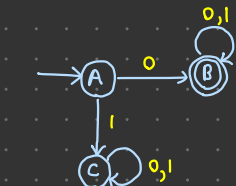
Transition Table

$(Q, \Sigma, q_0, F, \delta)$ Every DFA can be defined using these five tuples.

- Q = set of all states
- Σ = inputs
- q_0 = start state / initial state
- F = set of final states
- δ = transition function that maps from $Q \times \Sigma \rightarrow Q$

$L = \text{Set of all strings that start with '0'}$

$= \{0, 00, 01, 000, 010, 011, 0000, \dots\}$



dead state / trap state $\leftarrow$ trap here, cannot reach to final state

Eg, 001 $\checkmark$ Accepted

Initial state $A \xrightarrow{0} B \xrightarrow{0} B \xrightarrow{1} B$ - Final state

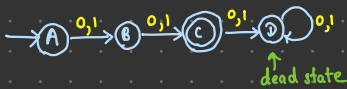
Eg, 101 $\times$ Not Accepted

Initial state $A \xrightarrow{1} C \xrightarrow{0} C \xrightarrow{1} C$ - Not Final state

Construct a DFA that accepts sets of all strings over $\{0,1\}$ of length 2.

$\Sigma = \{0,1\}$

$L = \{00, 01, 10, 11\}$



dead state

Eg, 00 $\checkmark$ Accepted

Initial state $A \xrightarrow{0} B \xrightarrow{0} C$ - Final state

Eg, 10 $\checkmark$ Accepted

Initial state $A \xrightarrow{1} B \xrightarrow{0} C$ - Final state

Eg, 001 $\times$ Not Accepted

Initial state $A \xrightarrow{0} B \xrightarrow{0} C \xrightarrow{1} D$ Not Final state

Eg, 1 $\times$ Not Accepted

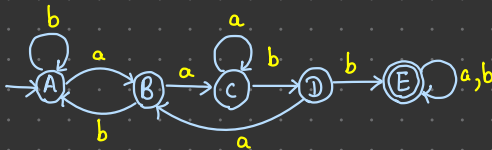
Initial state $A \xrightarrow{1} B$ Not Final state

Construct DFA that accepts any strings over $\{a,b\}$ that does not contain the string "aabb" in it.

$\Sigma = \{a,b\}$

Try to design a simpler problem

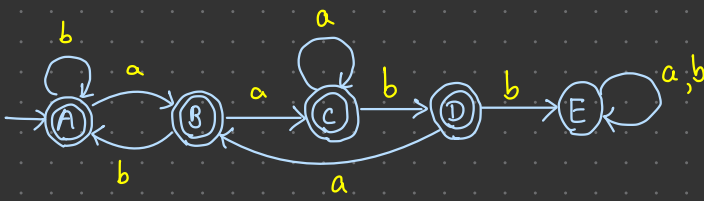
Let us construct a DFA that accepts all strings over $\{a,b\}$ that contains that string "aabb" in it.



- Flip the states

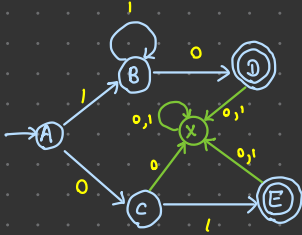
- Make the final state into non-final state

- Make the non-final state into final states



← DFA for all strings do not contain "acabb"

How to figure out what a DFA recognizes?



10 ✓
11...0 ✓
01 ✓

$L = \{ \text{accepts the string '01' or a string of at least one '1' followed by a '0'} \}$

Eg: 001, 010, 011, 1101, 1100

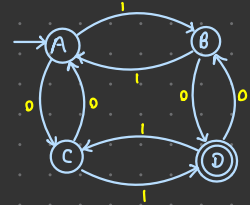
✗ ✗ ✗ ✗ ✗

Non-deterministic Finite Automata

Deterministic Finite Automata



Determinism *already determined*



- In DFA, given current state we know what the next state will be

- has only unique next state

- has no choice or randomness

- simple and easy to design

Non-deterministic Finite Automata

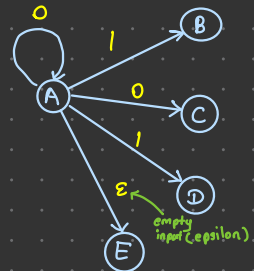


Non-determinism

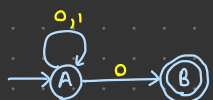
- In NFA, given the current state there could be multiple next states

- Next state may be chosen at random.

- or all the next states may be chosen in parallel.



NFA - Formal Definition



$L = \{ \text{Set of all strings that end with 0} \}$

NFA မှ next state ဘယ်လိုယူသွားရဲနဲ့

DFA မှာမရှိပါက complete မပြုမိဘူး

$(Q, \Sigma, q_0, F, \delta)$

Q = set of all states = $\{A, B\}$

Σ = inputs = $\{0, 1\}$

q_0 = start state / initial state = A

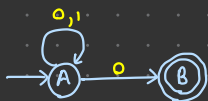
F = set of final states = $\{B\}$

$\delta = Q \times \Sigma \rightarrow$ = 2 $\leftarrow$ # states possibilities

state	input	
$A \times 0$		$\rightarrow A$
$A \times 0$		$\rightarrow B$
$A \times 1$		$\rightarrow A$
$B \times 1$		$\rightarrow \emptyset$
$B \times 0$		$\rightarrow \emptyset$

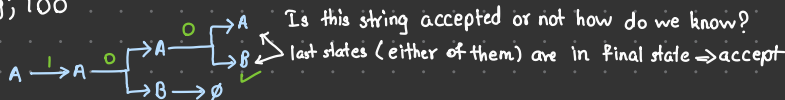
$A \xrightarrow{1} A, B, AB, \emptyset$ $\leftarrow$ 4 possibilities $2 \cdot 1 = 2^2$
 Suppose 3 states A, B, C $\uparrow$ 3 states $2 \cdot 1 = 2^2$
 $A \xrightarrow{1} A, B, C, AB, AC, BC, ABC, \emptyset$ $\leftarrow$ 8 possibilities $2 \cdot 3 = 2^3$

NFA - Example-1



$L = \{ \text{Set of all strings that end with 0} \}$

Eg; 100



Eg; 01



Example 2

$L = \{ \text{set of all strings that starts with 0} \}$

$= \{ 0, 00, 01, 000, \dots \}$



NFA

Eg; 001

$A \xrightarrow{0} B \xrightarrow{0} B \xrightarrow{1} B$ $\checkmark$ accepted

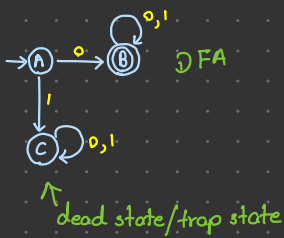
Eg; 101

$A \xrightarrow{1} \emptyset$ $\times$ not accepted

Dead Configuration

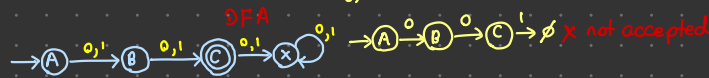
$\leftarrow$ in DFA we don't have dead configuration

(need to define dead state explicitly)



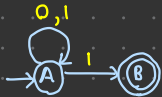
Construct a NFA that accepts set of all strings over $\{0,1\}$ of length 2

$$L = \{00, 01, 11, 10\} \quad \Sigma = \{0, 1\}$$



Ex 1) $L_1 = \{\text{set of all strings that ends with '1'}\}$

$$L_1 = \{1, 01, 11, 001, 011, 111, 101, \dots\}$$



Ex 2) $L_2 = \{\text{set of all strings that contain '0'}\}$

$$L_2 = \{0, 01, 10, 00, \dots\}$$



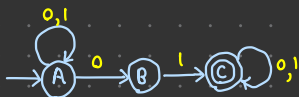
Ex 3) $L_3 = \{\text{set of all strings that starts with '10'}\}$

$$L_3 = \{10, 100, 101, 1000, 1010, 1011, \dots\}$$



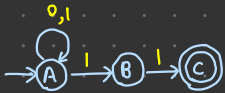
Ex 4) $L_4 = \{\text{set of all strings that contain '01'}\}$

$$L_4 = \{01, 001, 0010, 101, 1010, 1011, \dots\}$$



Ex 5) $L_5 = \{ \text{set of all strings end with '11'} \}$

$L_5 = \{ 11, 011, 111, 0011, 0111, 1111, \dots \}$



DFA to NFA

→ Every DFA is an NFA, but not vice versa

→ But there is an equivalent DFA for every NFA

DFA

$$\delta = Q \times \Sigma \rightarrow Q$$

NFA

$$\delta = Q \times \Sigma \rightarrow 2^Q$$

is a subset of

$L = \{ \text{set of all strings over } (0,1) \text{ that starts with '0'} \}$

$\Sigma = \{ 0, 1 \}$

$L = \{ 0, 00, 01, 000, 010, 011, \dots \}$

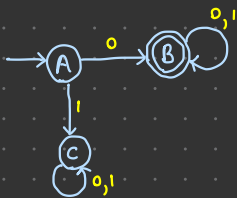
NFA



State Transition Table

	0	1
A	B	$\emptyset$
B	B	B

DFA



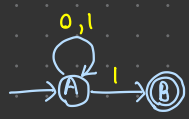
	0	1
A	B	C
B	B	B
C	C	C

NFA to DFA

$L = \{ \text{set of all strings over } (0,1) \text{ that ends with '1'} \}$

$L = \{ 1, 01, 11, 001, 011, 111, 101, \dots \}$

NFA

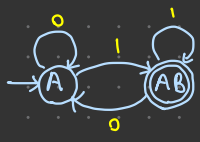


state Transition Table

	0	1
A	{A}	{A, B}
B	$\emptyset$	$\emptyset$

Subset Construction Method

DFA



$A \cup B$

	0	1
A	{A}	{AB} ← single state
AB	{A}	{AB}

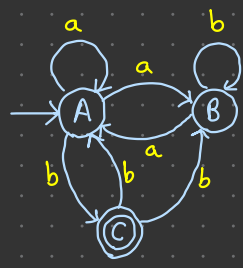
Example 2

Find the equivalent DFA for NFA given by $M = [Q, \Sigma, \delta, A, \{C\}]$ where δ is given by:

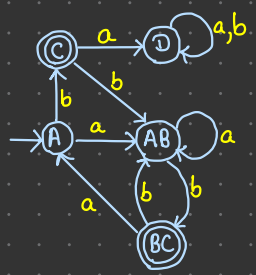
← given

	a	b
→ A	A, B	C
B	A	B
C	-	A, B

Final state in NFA



	a	b
→ A	AB	C
AB	AB	BC
BC	A	AB
C	D	AB
D	D	D



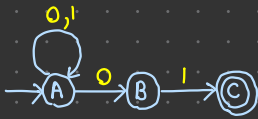
all the states that contain C are final states

Example 3

Given below is the NFA for a language

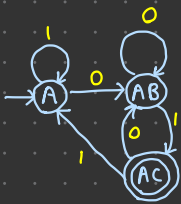
$L = \{ \text{set of all strings over } (0,1) \text{ that ends with '01'} \}$. Construct its equivalent DFA

NFA



	0	1
→ A	A, B	A
B	∅	C
C	∅	∅

DFA



	0	1
→ A	AB	A
AB	AB	AC
AC	AB	A

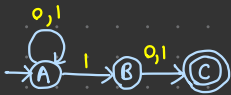
eg; 1101 ✓
1110 ✗

Example 4

Design a NFA for a language that accepts all strings over $\{0,1\}$ in which the second last symbol is always '1'. Then convert it to its equivalent DFA.

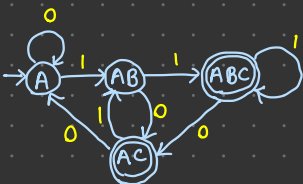
$\Sigma = \{0,1\}$, $L = \{10, 11, 010, 011, 111, 0010, \dots\}$

NFA



	0	1
→ A	A	A, B
B	C	C
C	∅	∅

DFA

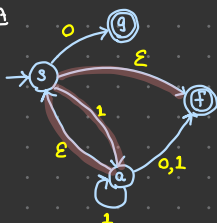


	0	1
→ A	A	AB
AB	AC	ABC
AC	A	ABC
ABC	AC	ABC

NFA to DFA

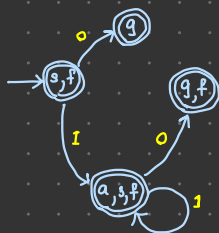
DFA start state: NFA start state q and all other NFA states reachable from q using ϵ transition

NFA



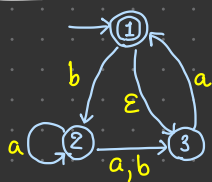
DFA State	0	1
$\rightarrow \{s, f\}$	$\{q\}$	$\{a, s, f\}$
$\{q\}$	—	—
$\{a, s, f\}$	$\{q, f\}$	$\{a, s, f\}$
$\{q, f\}$	—	—

DFA



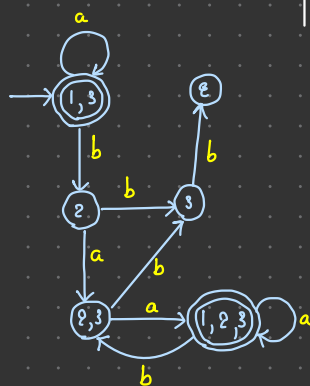
DFA Final state: Any states containing any NFA final states

NFA

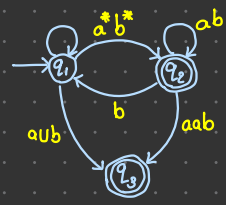


DFA state	a	b
$\rightarrow \{1, 3\}$	$\{1, 3\}$	$\{2\}$
$\{2\}$	$\{2, 3\}$	$\{3\}$
$\{2, 3\}$	$\{1, 2, 3\}$	$\{3\}$
$\{3\}$	$\{1, 3\}$	ϵ
$\{1, 2, 3\}$	$\{1, 2, 3\}$	$\{2, 3\}$

DFA



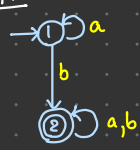
Generalized NFA



Regular expressions
as transition labels

NFA → GNFA → Regular Expression

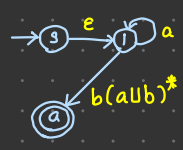
NFA



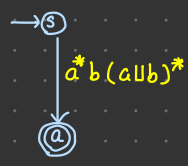
GNFA



Remove state 2

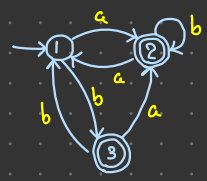


Remove state 1

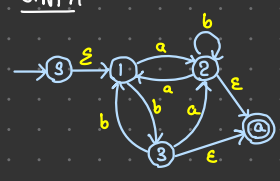


Regular Expression $\Rightarrow a^*b(aub)^*$

NFA



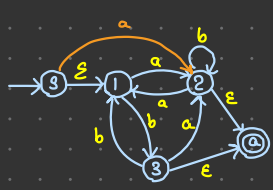
GNFA



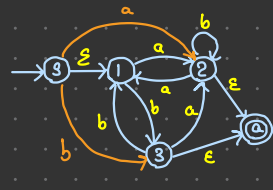
- $3 \rightarrow 1 \rightarrow 2$
- $3 \rightarrow 1 \rightarrow 3$
- $2 \rightarrow 1 \rightarrow 3$
- $2 \rightarrow 1 \rightarrow 2$
- $3 \rightarrow 1 \rightarrow 2$
- $3 \rightarrow 1 \rightarrow 3$

Removing state 1

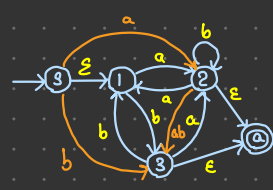
$3 \rightarrow 1 \rightarrow 2$



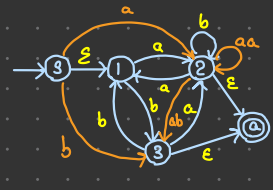
$3 \rightarrow 1 \rightarrow 3$



$2 \rightarrow 1 \rightarrow 3$



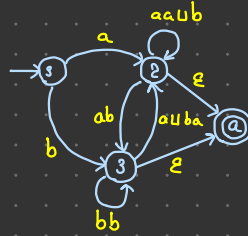
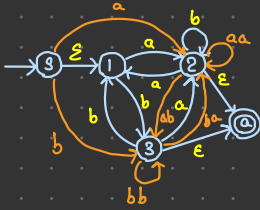
$2 \rightarrow 1 \rightarrow 2$



$$3 \rightarrow 1 \rightarrow 2$$



$$3 \rightarrow 1 \rightarrow 3$$



$$3 \rightarrow 2 \rightarrow a$$

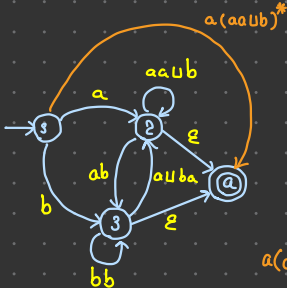
$$3 \rightarrow 2 \rightarrow 3$$

$$3 \rightarrow 2 \rightarrow a$$

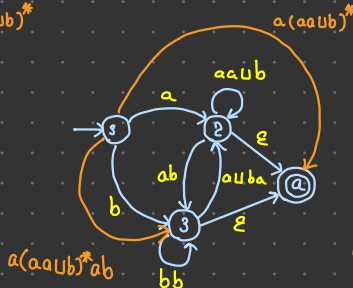
$$3 \rightarrow 2 \rightarrow 3$$

Remove state 2

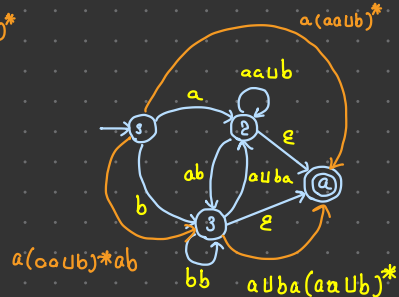
$$3 \rightarrow 2 \rightarrow a$$



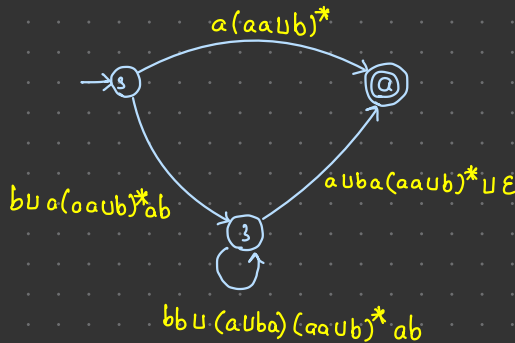
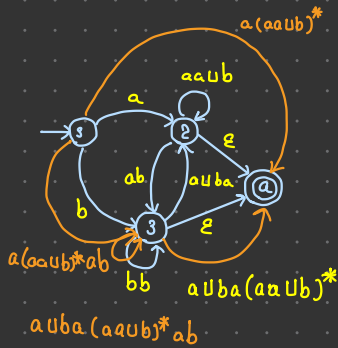
$$3 \rightarrow 2 \rightarrow 3$$



$$3 \rightarrow 2 \rightarrow a$$



$$3 \rightarrow 2 \rightarrow 3$$

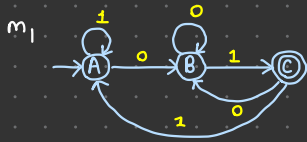


Remove state 3



$$a(aaUb)^*b \cup a(aaUb)^*ab(bb \cup aUba)(aaUb)^*ab)^*aUba(aaUb)^* \cup \epsilon$$

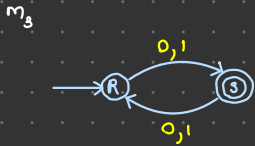
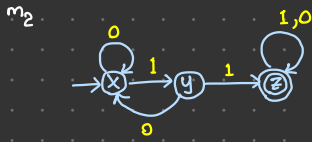
Union, Intersection and Complement



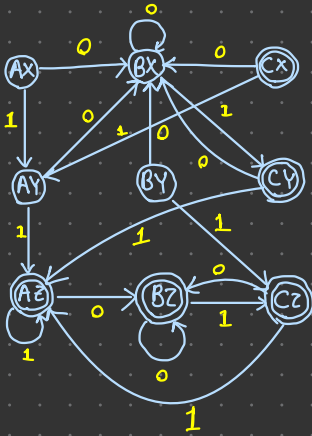
Draw FAs recognizing the following languages

a/ $L_1 \cup L_2$

b/ $L_1 \cap L_3$



a/ $L_1 \cup L_2 = \{AX, AY, AZ, BX, BY, BZ, CX, CY, CZ\}$



$L_1 = \{w \in \{0,1\}^* : w \text{ starts with } 0\}$

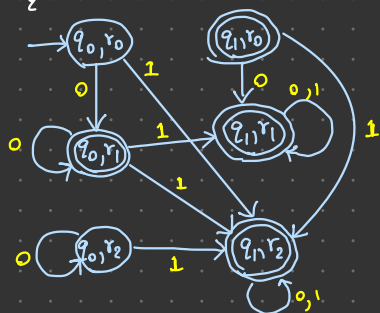
$L_2 = \{w \in \{0,1\}^* : w \text{ contains } 1\}$

Construct a DFA for $L_1 \cup L_2$ using the product construction.

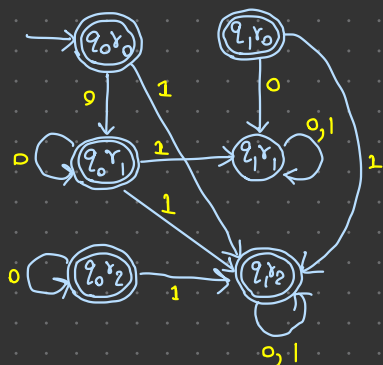
Make a DFA for L_1, L_2 .



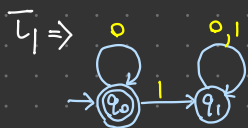
$$L_1 \cup L_2$$



$$\overline{L_1} \cup \overline{L_2}$$



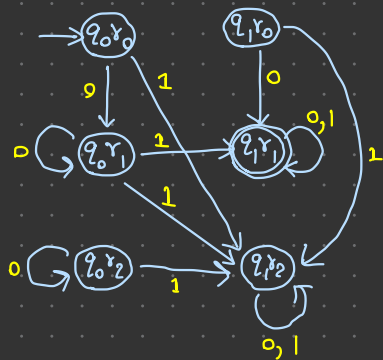
$$L_1 \cap L_2 = \overline{\overline{L_1} \cup \overline{L_2}}$$



$$\overline{L_2} \Rightarrow$$



$$\overline{\overline{L_1} \cup \overline{L_2}} = L_1 \cap L_2$$



Regular languages

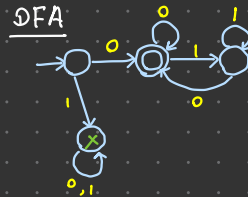
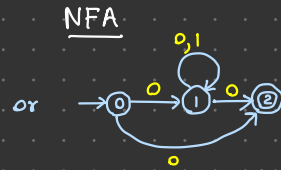
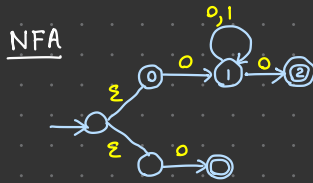
- A regular language is a language recognizable by some finite automaton.
- The class of regular languages is closed under (Example L_1, L_2 are regular)
 - Union: $L_1 \cup L_2$ is also regular
 - Concatenation: $L_1 L_2$ is also regular
 - Star: L^* is also regular
- The class of regular languages is also closed under
 - Intersection: $L_1 \cap L_2$
 - Complementation: $L_1^c = \Sigma^* - L_1$

Suppose $\Sigma = \{0, 1\}$

L is the language consisting of string beginning and ending with 0.

$$L = \{w \in \Sigma^* \mid w[0] = w[-1] = 0\}$$

$$L = \{0, 00, 000, 010, 0000, 0100, 0010, 0110, \dots\}$$



Reg Ex

$$L = L(0 \cup 0 \Sigma^* 0)$$

$$\alpha = 0 \cup 0 \Sigma^* 0 \leftarrow \text{string begin with 0 or end with 0 including 0}$$

$L(\alpha)$ is regular

$\overline{L(\alpha)}$ is also describable by some expression

$$\overline{L(\alpha)} \text{ is also regular} \Rightarrow \overline{L(\alpha)} = L(\text{---})$$

in regular expression $\cap, \overline{\text{---}}$ are not allowed.

only $\cup, \circ, *$ are allowed.

union concatenation star

$$L(\alpha) = (1 \Sigma^* \cup \Sigma^* 1 \cup \epsilon)$$

↑
string starts with 1 or end with 1 or empty string

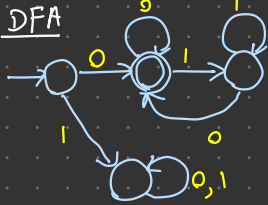
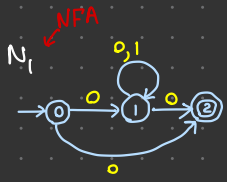
NFA N

complement
NFA N'

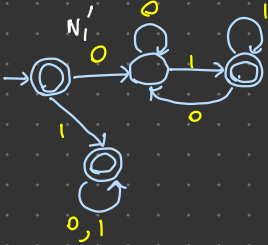
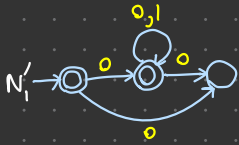
Complement of regular expression $\cap, \overline{\text{---}}$ given language of NFA of complement

NFA complement of regular expression on complement of given regular expression

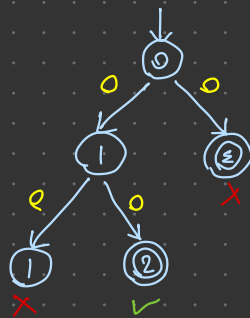
Complement of automaton (swapping accepting state and non-accepting state) (DFA)



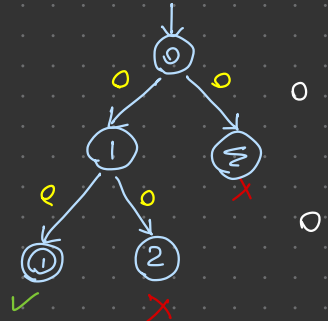
only work on DFA
so, NFA complement gives $\epsilon \delta$
DFA $\epsilon \delta$: $\epsilon \delta$ swap



$L(N_1) \neq L(N'_1)$



computation tree of N_1
accepts string 00

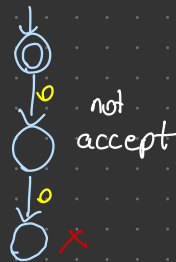
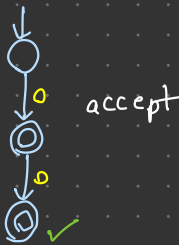
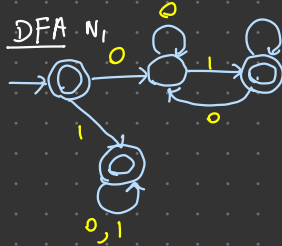
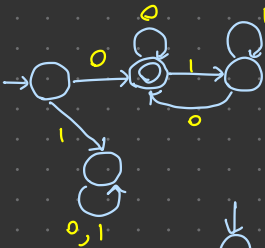


computation tree of N'_1 still
accepts string 00

bec some string may have many accepting paths

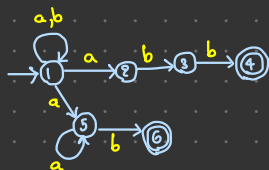
But if it is in DFA

DFA N_1



Regular Expression $\rightarrow$ Finite Automata

$(ab)^*(abb|a^+b)$



Pumping lemma

Lemma $\Rightarrow$ mini theorem

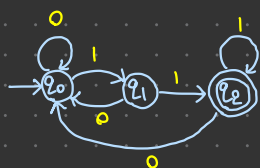
If A is a regular language, there is a pumping length p , where if s is in a string in A that is at least length p , then s may be divided into 3 pieces, $s = xyz$, satisfying the following conditions:

1. for each $i \geq 0$, $xy^iz \in A$
2. $|y| > 0$
3. $|xy| \leq p$

"if a language is regular, every string in the language will have a section that can be repeated or pumped any number of times and still will be in that language"

All strings in the language can be repeated — "pumped" — if they are at least as long as a certain length (pumping length)

$A = \{ \text{all strings that end in } 11 \}$



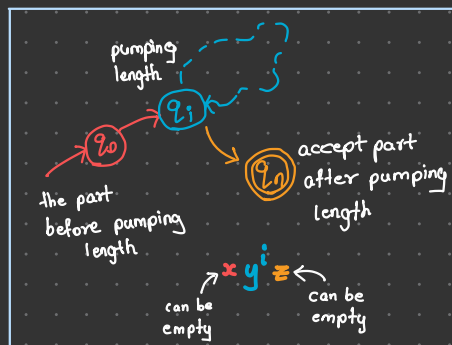
pumping length, $p = 3$ ← number of states

↑ strings that are at least as long as p can be pumped

$x \in A$

011	$\rightarrow$	0000011
1011	$\rightarrow$	1010101011
01111	$\rightarrow$	000001111

$A = \{ \text{all strings that end in } 11 \}$



The length of y can be at most p and must occur within in the first p symbols.

01111 must be within first $p=3$ symbols
cannot pump

NFA and DFA have finite amount of memory!

So, for some languages, we cannot construct finite machine (NFA, DFA) for them.

These languages are non-regular languages.

Proving Non-regularity!

Proof by Contradiction!

Proving $B = \{0^n 1^n \mid n \geq 0\}$ is not regular.

Proof

1. Assume B is a regular language

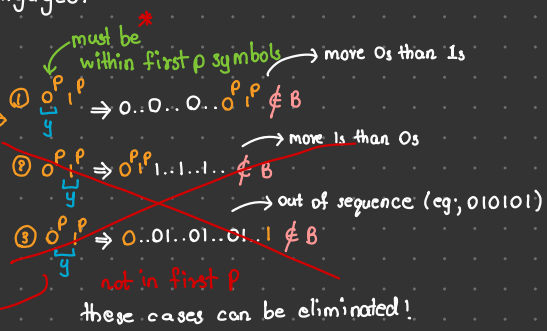
2. $0^p 1^p$

cannot pump

Not satisfy by pumping lemma!

Contradict regular languages' property!

So! B is not regular.



Proving Non-regularity by Pumping Lemma

- ① Assume language is regular
- ② Given p , choose a string that is at least length p
- ③ Show how pumping & splitting the string does NOT satisfy the pumping lemma.

Example

$$A = \{a^n b^n \mid n \geq 10\}$$

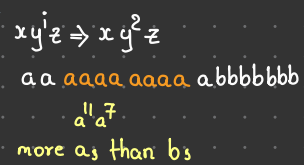
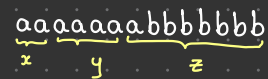
Assume A is regular.

pumping length = p

$$s = a^p b^p$$

Assume $p = 7$, $s = aaaaaaabbbbbbb$

case ① y is in 'a' part.



case ② y is in 'b' part.

aaaaaaabbbbbbb
 $\underbrace{\hspace{1.5cm}}_x \underbrace{\hspace{1.5cm}}_y \underbrace{\hspace{0.5cm}}_z$

case ③ y is in 'a' and 'b' part

aaaaaaabbbbbbb
 $\underbrace{\hspace{1.5cm}}_x \underbrace{\hspace{1.5cm}}_y \underbrace{\hspace{0.5cm}}_z$

$$xy^iz \Rightarrow xy^2z$$

aaaaaaabbbbbb
 a^7b^{11}

$$xy^iz \Rightarrow xy^2z$$

aaaaa aabb aabb bbbbbb

although 9a and 9b

doesn't follow pattern of $a^n b^n$

Example?

→ 0101

Prove $A = \{yy \mid y \in \{0,1\}^*\}$ is not regular.

By using Pumping Lemma,

Proof,

Assume that A is regular.

Then, it must have pumping length p. → need to choose wisely

Suppose, we choose the string $s = 0^p 1 0^p$ bcz, $0^p 0^p$ still $\in A$

Then we divide s into 3 parts x, y and z.

1^p still $\in A$

Assume $p = 7$,

Remember we want to prove non-regularity!

If a language has one such string that

does not satisfy Pumping Lemma  Not-regular Babe!

boom

00000010000001
 $\underbrace{\hspace{1.5cm}}_x \underbrace{\hspace{1.5cm}}_y \underbrace{\hspace{0.5cm}}_z$

$$xy^iz \Rightarrow xy^2z$$

000000000100000001 $\notin A$

doesn't follow pattern yy

000000100000001
 $\underbrace{\hspace{1.5cm}}_x \underbrace{\hspace{1.5cm}}_y \underbrace{\hspace{0.5cm}}_z$

$$|y| > 0 \checkmark$$

$$|xy| = 6$$

$$|xy| \leq p \checkmark$$

000000100000001
 $\underbrace{\hspace{1.5cm}}_x \underbrace{\hspace{1.5cm}}_y \underbrace{\hspace{0.5cm}}_z$

$$|xy| = 8 > 7 \times$$

Pumping lemma

suppose $0^n 1^n$
 $\downarrow$
 $0^p 1^p$

$$|xy| \leq p$$

$$|y| \geq 1$$

$$\begin{matrix} x = 0^\alpha & \alpha \geq 0 \\ y = 0^R & R \geq 1 \\ z = 0^{p-\alpha-R} 1^p \end{matrix}$$

$$\alpha + R \leq p$$

Choose i s.t. $xy^iz \notin L$

$$xy^iz = 0^\alpha 0^{iR} 0^{p-\alpha-R} 1^p$$

$$xy^iz = \begin{pmatrix} \alpha & i\beta & p-\alpha-\beta \\ 0 & 0 & 0 \end{pmatrix} \begin{pmatrix} \beta & p \\ 0 & 1 \end{pmatrix} \begin{pmatrix} p \\ 0 \end{pmatrix}$$

$$= \begin{pmatrix} \alpha & i\beta & p-\alpha-\beta \\ 0 & 0 & 0 \end{pmatrix} \begin{pmatrix} \beta & p \\ 0 & 1 \end{pmatrix} \begin{pmatrix} p \\ 0 \end{pmatrix}$$

$$= \begin{pmatrix} p+i\beta-\beta & p \\ 0 & 1 \end{pmatrix} \begin{pmatrix} p \\ 0 \end{pmatrix}$$

Choose i st $p+i\beta-\beta \neq p$

$$i\beta - \beta \neq 0$$

$$i\beta \neq \beta$$

$$i \neq 1$$

we pick $i=2 \Rightarrow xy^iz = \begin{pmatrix} p+2\beta-\beta & p \\ 0 & 1 \end{pmatrix} \begin{pmatrix} p \\ 0 \end{pmatrix}$

$$= \begin{pmatrix} p+\beta & p \\ 0 & 1 \end{pmatrix} \begin{pmatrix} p \\ 0 \end{pmatrix} \Rightarrow \text{contradict } \begin{pmatrix} p & p \\ 0 & 1 \end{pmatrix} \text{ pattern!}$$

$L = \{w \in \{0,1\}^* \mid w \text{ has same number of 0s, 1s}\}$

closure property technique

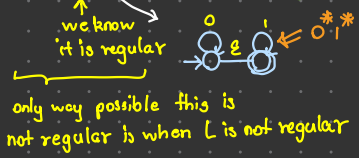
$$\begin{matrix} 0101 \in L \\ 0001 \notin L \end{matrix}$$

Assume $L \cap 0^*1^*$

Since regular languages are closed under intersection ($\cap$), $L \cap 0^*1^*$ must also be regular.

$0^*1^* \neq 0^n1^n$ *

$L \cap 0^*1^* = \{0^n1^n \mid n \geq 0\}$ ← proved above not regular



$L = \{01, 10, 0011, 1100, 1010, 0101, 1001, 0110, \dots\}$

$0^*1^* = \{\epsilon, 0, 1, 00, 01, 11, 000, 111, 011, 0000, 1111, 0111, 0011, 0001, \dots\}$

$0^n1^n = \{01, 0011, 000111, 00001111, \dots\}$

$\Rightarrow \therefore L \cap 0^*1^* = 0^n1^n$

$L = \{w \in \{0,1\}^* \mid w \text{ has more 1s than 0s}\}$

Suppose L is regular

$\Rightarrow$ exists a p for L .

← pumping length must be in first p .

Choose $w = 0^p 1^{p+1}$

xy^iz : $x = 0^a$ $a \geq 0$

$y = 0^b$ $b \geq 1$

$$z = 0^{p-d-R} 1^{p+1}$$

$$xy^iz \Rightarrow \cancel{0^p} \cancel{0^i} \cancel{0^R} \cancel{0^{p-d-R}} 1^{p+1}$$

$$= 0^{p+iR-R} 1^{p+1}$$

$$\text{In } L, \cancel{p+iR-R} < \cancel{p+1}$$

$$iR-R < 1$$

$$R(i-1) < 1$$

$$\text{choose } i=2 \gg 1$$

$$\boxed{0^{p+R} 1^{p+1}}$$

although we choose any other larger i still 0 will more than 1 when we pump

So, $\#0$ s will be more than $\#1$ s Not regular!

$$L = \{0^n 1^n \mid n \geq 0\}$$

Suppose L is regular.

$\Rightarrow$ exists p for L .

$$\text{Choose } w = 0^{2p} 1^p$$

$$xyz \Rightarrow x = 0^{\alpha} \quad \alpha \geq 0$$

$$y = 0^R \quad R \geq 1$$

$$z = 0^{2p-d-R} 1^p$$

$$xy^iz = \cancel{0^{\alpha}} \cancel{0^i} \cancel{0^R} \cancel{0^{2p-d-R}} 1^p$$

$$= 0^{2p+iR-R} 1^p$$

he said in language L $\therefore$

$$\text{In } L, \cancel{2p+iR-R} = \cancel{2p}$$

$$i=1$$

But he choose $i=2$ $\therefore$

$$= 0^{2p+R} 1^p$$

more 0s! not regular!

$$L = \{0^n 1 0^n \mid n \geq 0\}$$

Assume L is regular.

there exists p in L .

Choose a string $s = 0^p 1 0^p$

$$\begin{aligned} xyz &\Rightarrow x = 0^\alpha & \alpha \geq 0 \\ y &= 0^\beta & \beta \geq 1 \\ z &= 0^{p-\alpha-\beta} 1 0^p \end{aligned}$$

$$xy^i z \Rightarrow \begin{array}{c} \cancel{\alpha} \quad i\beta \quad p-\cancel{\alpha}-\beta \quad 1 \quad 0^p \\ 0 \quad 0 \quad 0 \quad 1 \quad 0^p \\ 0^{p+i\beta-\alpha} \quad 1 \quad 0^p \end{array}$$

In L , we choose $0^p 1 0^p$
 $\underbrace{\hspace{1.5cm}}$
 they have to be the same

$$\begin{aligned} \text{so, we need } \cancel{p} + i\beta - \cancel{\alpha} &= \cancel{p} \\ i\beta &= \beta \\ i &= 1 \end{aligned}$$

so, if we chose i any value other than 1 will make $0^p 1 0^p$ out of the pattern.

Suppose, we choose $i = 2$

$$\begin{array}{c} p+\beta \quad 1 \quad 0^p \\ 0 \quad 0 \quad 0 \end{array}$$

Contradiction! L is not regular.

$$L_1 = \{0^m 1^n 0^{m+n} \mid m, n \geq 0\}$$

$$L_2 = \{0^m 1^n 0^{m \cdot n} \mid m, n \geq 0\}$$

We make $L_3 = \{0^m 1^n 0^{f(m,n)} \mid m, n \geq 0\}$

Suppose L_3 is regular.

$\Rightarrow$ exists p for L_3

Choose $w = 0^p 1^p 0^{f(p,p)}$

$$xyz \Rightarrow x = 0^{\alpha} \quad \alpha \geq 0$$

$$y = 0^{\beta} \quad \beta \geq 1$$

$$z = 0^{p-\alpha-\beta} 0^{f(p,p)}$$

$$xy^iz \Rightarrow 0^{\alpha} 0^{\beta} 0^{p-\alpha-\beta} 0^{f(p,p)}$$

$$\Rightarrow 0^{p+i\beta-\beta} 0^{f(p,p)}$$

Choose i, f_1, f_2 s.t. $xy^iz \notin L_1, L_2$.

In $L_1 \Rightarrow p+i\beta-\beta + f_2 = p + f_2$

$$0^p 1^p 0^{p+f_2}$$

$$i\beta - \beta = 0$$

$$i = 1$$

when i is any other number rather than 1, will

make it out of pattern. let say $i=2$

$$0^p 1^p 0^p \quad 0^p 1^{p_2} \quad 0^{p+p_2}$$

move 0 here

In $L_2 \Rightarrow (p_1 + i\beta - \beta) \cdot p_2 = p_1 \cdot p_2$

$$0^p 1^p 0^{p \cdot p_2}$$

$$p_1 + i\beta - \beta = p_1$$

$$i\beta - \beta = 0$$

$$i = 1$$

choose $i=2$

$$0^p 1^p 0^p \quad 0^p 1^{p_2} \quad 0^{p \cdot p_2}$$

$$L = \{0^n 1^m \mid n \neq m\}$$

Suppose L is regular.

$$w = 0^{p+1} 1^p$$

it seems reasonable but it's not.

more zero than 1
pumping should in first p

use $\Rightarrow 0^{p \cdot p+1} 1^p$

Honey we will be tested!

p! trick

Instead, $w = 0^p 1^{p!}$

$$xyz \Rightarrow x = 0^{\alpha} \quad \alpha \geq 0$$

$$y = 0^{\beta} \quad \beta \geq 1$$

$$z = 0^{p-\alpha-\beta} 1^{p+p!}$$

$$xy^iz \Rightarrow 0^{\alpha} 0^{\beta} 0^{p-\alpha-\beta} 1^{p+p!}$$

$$= 0^{p+i\beta-\beta} 1^{p+p!}$$

Choose i s.t. $xy^iz \in L$

In $L \Rightarrow p+i\beta-\beta \neq p+p!$

$$i\beta - \beta \neq p!$$

$$\beta(i-1) \neq p!$$

Since, $\beta \geq 1$

$$1 \leq \beta \leq p$$

divisor
 $\downarrow$
 $\beta \mid p!$

$i \neq \frac{p!}{R} + 1$ not equal to 0, 1, 2, 3, 4

let just choose $i = \frac{p!}{R} + 1$

$$L = \{0^n 1^m \mid n < 3m\}$$

$$w = 0^{3p-1} 1^p$$

$$xyz \Rightarrow 0^i 0^R 0^{3p-1-R} 1^p$$

$$\Rightarrow 0^{3p+R-R} 1^p$$

$$3p+R-R < 3p$$

$$i < 1$$
 ← cannot be 0, 1, 2, 3, 4

choose $i = 2$

$$0^{3p+R} 1^p$$

more zero!

$$L = \{0^n 1^2 \mid n \geq 0\}$$

$$w = 0^p 1^2$$

$$xyz \Rightarrow 0^i 0^R 0^{p-R} 1^2$$

$$\Rightarrow 0^{p+R-R} 1^2$$

We have to choose i s.t. $xyz \in L$ since it's hard to choose

let suppose $i = 2$

$$\Rightarrow 0^{p^2+R} 1^2$$

In L , p^2+R is a perfect square.

but, $p^2 < p^2+R$ $R \geq 1$

and $p^2 < p^2+R \leq p^2+p$ p^2+2p+1

and $p^2 < p^2+R \leq p^2+p < (p+1)^2$

$$p^2 < \underbrace{p^2+R}_{\text{this string will never be a perfect square}} \leq p^2+p < \underbrace{p^2+2p+1}_{(p+1)^2}$$

this string will never be a perfect square $p^2, p^2+2p+1 = (p+1)^2$

$$L = \{0^{2^n} \mid n \geq 0\}$$

$$L = \{0, 00, 0000, 00000000, \dots\}$$

$$w = 0^{2^p}$$

$$0^{2^p+R}$$

$$0^{2^p+(i-1)R}$$

In L , $0^{2^p+(i-1)R}$ is a power of 2.

If $i = z$,
 $z^p < z^{p+R} < z^{p+P} < z^{p+1} \rightarrow z^{p+P}$
 $R > 1$ $R < P$ $P < z^P$

$z^p + R$ is strictly between consecutive power z ,

it can't be equal to either one of them. *Not regular!*

$$L = \{w \in \{0,1\}^* \mid w \text{ is not a palindrome}\}$$

$\rightarrow 0110, 1001, 010, 101, \dots$

$$w = 0^p 110^{p+P!}$$

$$xy^i z \Rightarrow 0^{p+iR-R} 110^{p+P!}$$

$$\text{In } L, p + (i-1)R \neq p + P!$$

$$(i-1)R \neq P! \quad \leftarrow P! \text{ trick}$$

$$i \neq \frac{P!}{R} + 1$$

$\rightarrow$ is integer
 $1 < R < P$

$$\text{Let chose } i = \frac{P!}{R} + 1$$

it means, when $i = \frac{P!}{R} + 1$, it is a palindrome!

Contradict!

$$L = \{0^m 1^n \mid \frac{m}{n} \text{ is an integer}\}$$

$\uparrow$
 $\times$ of 0s is multiple of $\times$ of 1s, $\times$ of 0s is divisibly $\times$ of 1s.

$$0^p 1^p \text{ integer } 059 \leftarrow \frac{m}{n}$$

$$\text{Choose } 0^{2P} 1^P$$

$$0^{4P+4R-R} 1^{2P}$$

$$\text{In } L, \frac{4P+(i-1)R}{2P} \text{ is an integer}$$

$$i=2$$

$$\frac{4P+R}{2P} \leftarrow 1 \leq R \leq P \text{ not integer}$$

